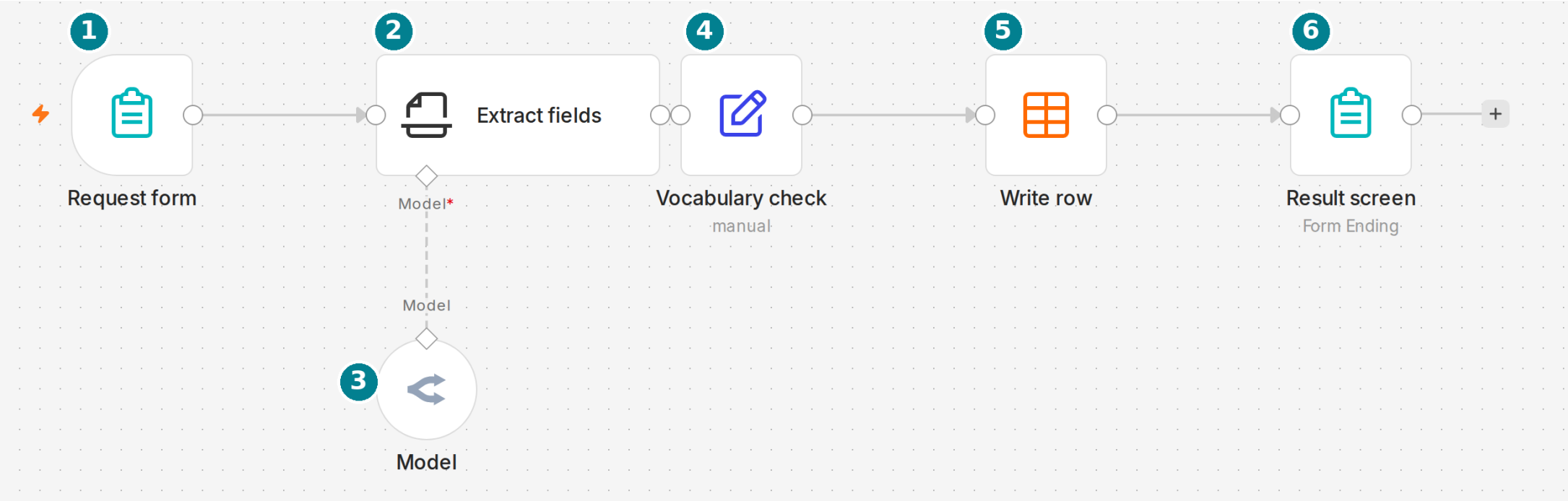
Где решает модель, а где должен — код

Процессы на больших языковых моделях: второе занятие дуги.

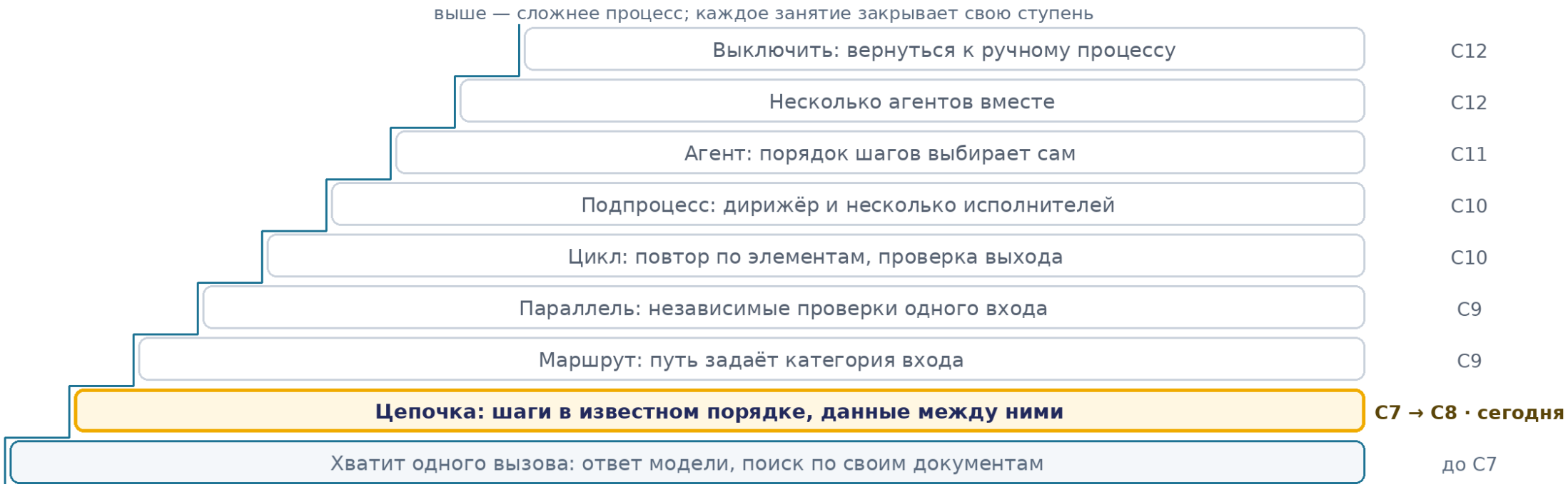
Входное состояние: процесс с прошлого занятия



1 форма — вход процесса · 2 извлечение — достаёт значения из текста · 3 модель — второй узел шага
4 сверка со списком · 5 запись строки · 6 экран результата — печатает всю дорогу

Это настоящий холст n8n, а не схема. У нас есть процесс по классификации входящих обращений — шесть узлов, тот же порядок, те же имена, что и в конце прошлого занятия.

Та же ступень: цепочка



Золотая полоса стоит там же, где и в прошлый раз. Сегодняшние четыре кейса — не подъём выше, а то, что происходит внутри одной и той же ступени, когда к цепочке добавляют новые шаги.

Ось занятия: где решает модель, где должен — код

ВЫГЛЯДИТ НАДЁЖНЫМ	ЧЕМ ОКАЗАЛСЯ
сообщение ушло координатору, цепочка короче и дешевле без LLM	
заявка про тот же вопрос пришла снова — обработали как новую	
в промте написано «не возвращай больше суммы X»	
картинка собралась и ушла в Telegram	

Правая колонка закроется не вся разом в конце, а по одной строке — сразу после того кейса, который её закрывает. И каждый раз не моим рассказом, а тем, что мы увидим на экране.

Кейс 1. Уведомления

«Пришлите мне в Telegram одну строку: кто написал и про что — чтобы не лезть в таблицу на каждую заявку.»

Уведомления | Проверка на повтор | Одобрение возврата | Карточка в Telegram

Экран результата и многотемная заявка

Учебный центр, тот же процесс, что заработал на прошлом занятии: форма → извлечение полей → запись в таблицу с темой, дополнительными темами и срочностью → экран результата. Дальше с этим экраном работает Ольга, администратор — двадцать заявок в день, и большую часть она разбирает по экрану результата спокойно: тема и срочность уже там, кому передать — понятно сразу.

1. Раз в несколько заявок текст короче обычного не получается: человек упомянул не только основной вопрос, а что-то ещё — спросил про оплату, попросил перенести занятие, уточнил расписание. Поле «дополнительные темы» в таблице это ловит и честно записывает.
2. Экран результата Ольга открывает не на каждую заявку сразу — за день их двадцать, и открывать каждую по готовности было бы ровно той же работой руками, которую процесс должен был снять. Она проверяет пачками, несколько раз за смену.
3. Если многотемная заявка пришла между проверками, она лежит в таблице с правильными полями — и ничем не выделяется среди обычных заявок, пока Ольга не откроет экран сама и не прочитает именно эту строку.
4. К вечеру, если день был плотным, она честно признаёт: часть многотемных заявок она заметила не в момент прихода, а при следующей пачке проверки — иногда через несколько часов.

Чем платит учебный центр за задержку. Многотемная заявка почти всегда срочнее обычной: человек не просто спрашивает про курс, он ещё и хочет перенести оплаченное занятие на сегодня или уточнить то, что должно случиться до конца дня. Пачка проверки раз в несколько часов превращает именно эту срочность в обычную очередь.

Ольга формулирует желание предельно конкретно: «Пришлите мне в Telegram одну строку на каждую такую заявку — кто написал, по какой теме и что ещё упомянул, — чтобы не идти проверять экран результата из-за одной многотемной заявки среди двадцати обычных.»

Как собрать уведомление из трёх значений

Как собрать текст уведомления из этих трёх значений?

модель пишет весь текст уведомления целиком, свободно

заполняем готовую форму сообщения алгоритмом целиком — только подстановка значений

то, что можно закрыть алгоритмом, закрываем им; то, что требует интерпретации текста заявки, на то отдельно зовём модель

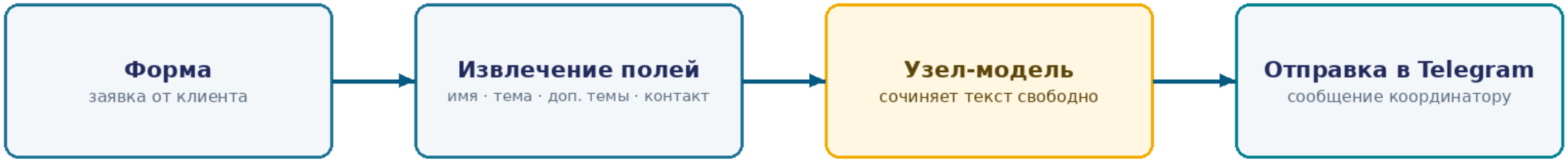
берём алгоритмически собранный текст как основу и добавляем к нему одну фразу тона от модели поверх

Что делает каждое действие

ДЕЙСТВИЕ	КАК ДЕЛАЕТ	ПРЕИМУЩЕСТВО	НЕДОСТАТКИ
модель пишет весь текст целиком	получает три уже извлечённых значения и сочиняет текст уведомления свободно, без заданной формы	текст звучит живо, не казённой фразой — умеет писать по-человечески	значения уже извлечены — интерпретировать нечего; текст собирается заново на каждом прогоне, с риском вписать деталь, которой в заявке не было
алгоритм заполняет форму целиком	подставляет три уже структурированных значения в один раз заданный шаблон текста, без обращения к модели	результат детерминированный: совпадает с заданным примером текста на любой заявке, не только на удачной	нет пространства для вариативности: если форма сообщения когда-нибудь должна будет меняться по смыслу, а не только по значениям, выражение придётся переписывать руками
алгоритм + модель на то, что требует интерпретации	структурированные значения подставляются алгоритмом; то, что осталось неструктурированным и требует интерпретации текста заявки, отдельно передаётся модели	работало бы честно при реальном смешанном случае — когда хотя бы одно значение правда требует интерпретации, а не просто известно	для этой конкретной потребности интерпретировать нечего: все три значения, включая срочность, уже извлечены структурно на прошлом занятии — модели здесь просто не находится работы
алгоритм + тон модели поверх	алгоритмический костяк текста собирается подстановкой значений, модель добавляет сверху одну короткую фразу тона или комментария	масштаб свободы меньше, чем у первого действия — риск и цена вызова модели пропорционально ниже	риск той же природы, что у первого действия, просто меньшего размера: модель всё равно генерирует текст и может вписать в него деталь, которой не было

Архитектура: шаг-модель сочиняет текст

Решение с LLM: модель сочиняет текст



Риск: модель может добавить деталь, которой не было в заявке.

Плюс цена и задержка вызова модели на каждый прогон.

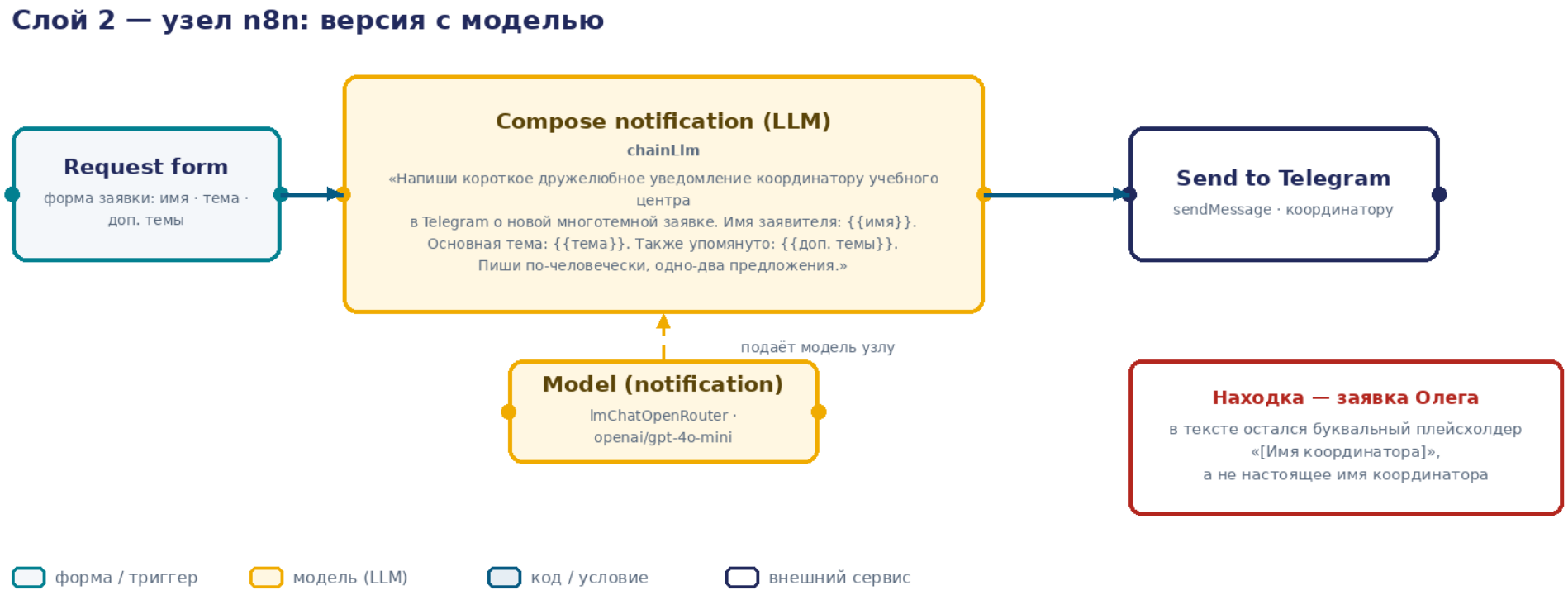
Четыре блока: форма → извлечение полей → шаг, вызывающий модель на каждый прогон → Telegram. Решение о форме текста принимается заново при каждой заявке.

Архитектура: шаг-шаблон подставляет значения



Та же цепочка: форма → извлечение полей → подстановка значений в фиксированную форму → Telegram. Решение о форме текста принято один раз, заранее, а не на каждом прогоне.

Настройка узла: версия с моделью



Программно нарисованная схема в духе n8n — холст недостижим в этой среде ([#233-8]); имена узлов и значения — настоящие, со стенда.

Узел «Compose notification (LLM)» получает три уже извлечённых значения и инструкцию «напиши короткое дружелюбное уведомление... одно-два предложения, пиши по-человечески». Текст уходит прямиком в узел отправки в Telegram.

Как сообщение выглядит у Ольги в телефоне

Telegram, чат «Уведомления учебного центра».

Новая заявка от Марии: тема «курс», также упомянуто: оплата.

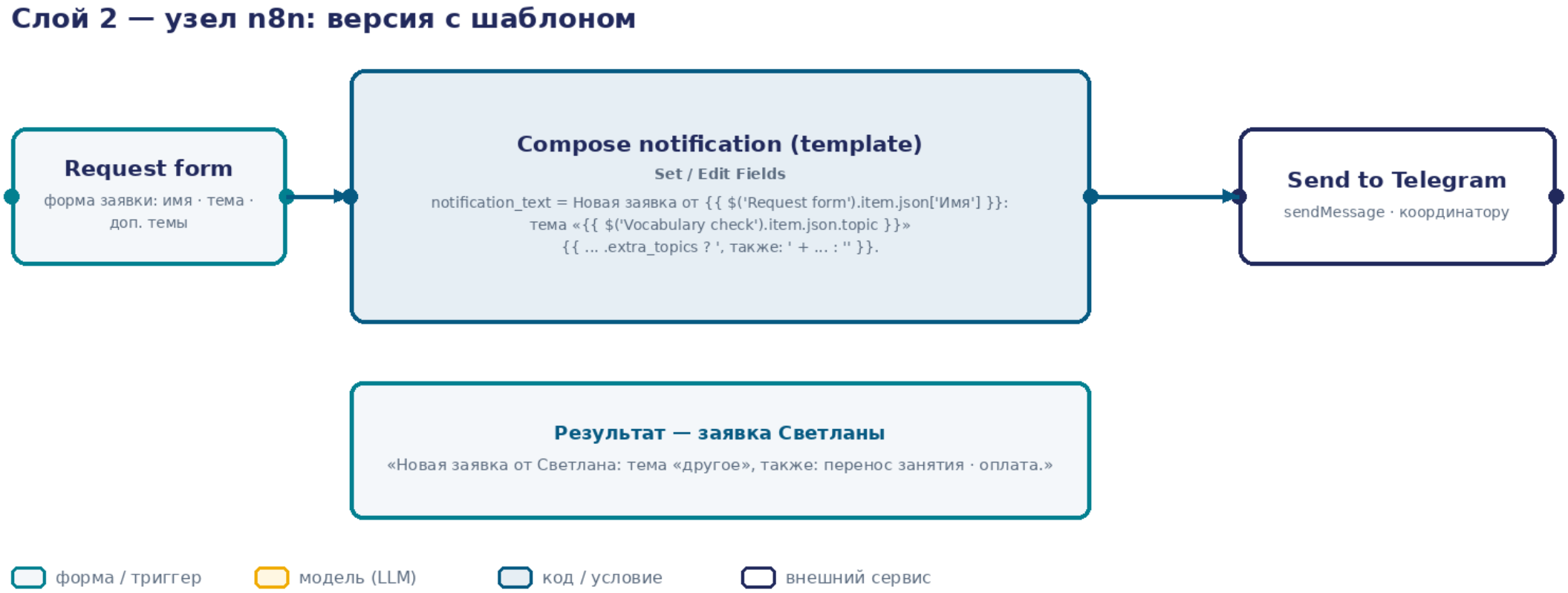
Ровно то, что Ольга описала в прошлой сцене, слово в слово: кто написал, по какой теме и что упомянул ещё — одна строка, без необходимости открывать экран результата.

Что реально получилось на двух прогонах

ТЕКСТ ЗАЯВКИ	ЧТО ПОЛУЧИЛОСЬ И ПОЧЕМУ
Мария: «хочу на курс по аналитике и ещё спросить про оплату, перезвоните сегодня»	Пришло: «Новая заявка от Марии: тема «курс», также упомянуто: оплата.» Текст свободный, но совпал по смыслу с целью — модели хватило инструкции и трёх значений, интерпретировать было действительно просто.
Олег: «хочу перенести занятие на пятницу и спросить про оплату курса»	Пришло: «Привет, **[Имя координатора]**! У нас новая многотемная заявка от Олега по вопросу переноса занятия и об оплате.» Модель вписала в текст буквальный кусок собственного шаблона-инструкции — слова, адресованные ей самой, а не Ольге. Узел не упал, сообщение доставлено, статус зелёный: расхождение видно только по содержанию.

Проверка форматом этого не ловит. Оба прогона технически успешны: оба узла отработали, оба сообщения дошли. Единственный способ заметить разницу — прочитать сам текст, который получил живой человек.

Настройка узла: версия с шаблоном



Узел «Compose notification (template)» собирает текст одним выражением: «Новая заявка от [имя]: тема «[тема]»[, также: [доп. темы]].» — значения подставляются в заранее написанную форму. Результат уходит в тот же узел отправки, что и у версии с моделью.

Не каждый текстовый вывод — это генерация

ПРИНЦИП	**ЕСЛИ ВСЕ НУЖНЫЕ ЗНАЧЕНИЯ УЖЕ СТРУКТУРИРОВАНЫ И ФОРМА СООБЩЕНИЯ НЕ МЕНЯЕТСЯ ПО СУТИ — ЭТО ЗАДАЧА ФОРМАТИРОВАНИЯ, А НЕ ЗАДАЧА МОДЕЛИ.**
Класс задачи	Сборка сообщения из уже извлечённых значений по фиксированной форме.

Когда нужна модель, когда достаточно шаблона

НЮАНС

Лишний шаг — риск, а не только цена

Шаблон не навсегда безопасен

ЧТО ЭТО ЗНАЧИТ

Каждый вызов модели — точка, где может появиться галлюцинация, даже в тривиальной задаче форматирования.

Если форма сообщения начнёт требовать реальной вариативности (разные языки, разная степень детализации), шаблон придётся переписывать руками на каждый новый случай — цена другого типа, инженерная.

Критерий. Модель нужна при реальной вариативности контента или когда значение требует интерпретации текста, которой не было на предыдущем шаге. Шаблон достаточен, когда форма сообщения одна и все значения уже структурированы.

Первая строка оси закрыта. Сообщение действительно ушло координатору — а цепочка оказалась короче и дешевле без узла модели. LLM здесь был просто лишним шагом: то же самое получилось шаблоном, без его цены и риска.

Кейс 2. Проверка на повтор

«Это ведь тот же вопрос, с которым он писал два часа назад?»

Заявки дублируются

1. Понедельник, утро. Ольга, администратор учебного центра, открывает таблицу заявок и видит вопрос от клиента: «когда можно оплатить занятие?» Она отвечает.
2. Через два часа — вторая заявка от того же контакта: «как мне внести оплату за курс». По смыслу это тот же вопрос. Ольга не узнаёт перефразировку и обрабатывает её как новую: отвечает ещё раз.
3. Клиент получает два разных ответа от центра с разницей в два часа — и это выглядит так, будто первый ответ потерялся.

Это не редкость. По собственной оценке Ольги, такие повторы случаются примерно 10 раз в день: кто-то забыл, что уже спрашивал, кто-то не увидел ответа и переспросил другими словами.

Цена ошибки не абстрактная. Двойная работа администратора — плюс недоверие клиента: «они не видят, что я уже спрашивал?»

Как построить проверку на повтор

Как построить проверку на повтор?

- сравнивать тексты буквально — точное совпадение после нормализации
- сравнивать тексты по общим словам — классический векторный подход с порогом
- спросить у модели общего назначения, один ли это вопрос, просто сформулированный иначе
- спросить у специализированного сервиса сравнения вопросов, а не у модели общего назначения
- сначала проверить точное совпадение дёшево, а дальше пускать только то, что осталось

Что поймала каждая схема

20 тестовых пар: 10 настоящих дублей (5 буквальных повторов + 5 перефразировок из жизни центра) и 10 заведомо разных вопросов, 3 из них — на близкую тему, для честной проверки ложных срабатываний.

СХЕМА	ВЕРНО ИЗ 20	ЧТО ПОЙМАЛА	ЧТО ПРОПУСТИЛА / ГДЕ ОШИБЛАСЬ	СТОИМОСТЬ И СКОРОСТЬ
Хэш (точное совпадение)	**15/20**	все 5 повторов, все 10 разных	все 5 перефразировок (0/5)	дёшево и мгновенно — без вызова модели
TF-IDF + косинус (порог 0,3)	**13/20** — хуже хэша	повторы, часть разных вопросов	те же 5 перефразировок + 2 ложных срабатывания на близких темах (хэш здесь не ошибается)	дёшево и мгновенно — без вызова модели
Модель-судья (gpt-4o-mini)	**17/20**	все 10 дублей	3 ложных срабатывания, включая пару без общих слов	~$0,15/млн вход · $0,60/млн выход — доли цента/вызов (оценка по прайсу)
Сервис jev	**20/20**	все 20 без ошибок	чистое разделение, без пересечений	$0,042/млн вход, $0 выход — оценка по прайсу

Честная находка. При пороге 0,3 классический подход не просто не даёт прироста — добавляет 2 ложных срабатывания там, где хэш не ошибается в принципе. Поднять порог до значения, которое убрало бы эти ложные срабатывания, не помогает поймать ни одну перефразировку: лучшее, что даёт настройка порога на этом наборе, — сравняться с хэшем, а не превзойти его. И вторая честная находка: лучший результат на этом тесте дал не самый очевидный кандидат — узкий специализированный сервис обошёл универсальную модель-судью, и обошёлся дешевле неё, но одного измерения на 20 парах недостаточно, чтобы на этом основании менять уже работающую архитектуру.

Рекомендация: хэш первым, jev — на оставшееся



Фигура: шаг 1 — дешёвый механический предфильтр, берёт на себя буквальные повторы бесплатно. Шаг 2 — проверка смыслом, рекомендация по измерению — специализированный сервис (jev), точнее и дешевле модели общего назначения на этом тесте.

Честная оговорка — на самой схеме. Рекомендация по одному тесту на 20 парах одной тематики — не повод слепо менять уже собранную архитектуру. Цепочка, показанная на стенде дальше, пока использует модель-судью на втором шаге, а не jev.

Настройка: нормализация и точное совпадение



Программно нарисованная схема в духе n8n — холст недостижим в этой среде ([#233-8]); имена узлов и значения — настоящие, со стенда.

Узел `Normalize + hash (Set)`. Берёт текст заявки, приводит к нижнему регистру, убирает пробелы по краям, схлопывает повторные пробелы внутри: `normalized_hash = текст.toLowerCase().trim().replace(/\s+/g, ' ')`. Рядом — тот же расчёт для недавнего сообщения того же контакта.

Узел `Exact duplicate in window? (IF)`. Условие: `normalized_hash` строго равен хэшу недавнего обращения — строковое сравнение, оба значения уже в нижнем регистре. Совпало → повтор найден, гасится бесплатно, без записи. Не совпало → заявка идёт дальше по цепочке.

Настройка: узел-модель спрашивает напрямую



Программно нарисованная схема в духе n8n — холст недостижим в этой среде ([#233-8]); имена узлов и значения — настоящие, со стенда.

Промпт узла `Same question, reworded?` (`chainLlm`). «Это новое сообщение: «{{новый текст}}». Недавнее сообщение того же контакта: «{{старый текст}}». Это тот же вопрос, просто сформулированный иначе? Ответь строго «да» или «нет».»

ПРОГОН	ЧТО ПОДАЛИ	ЧТО ПОЛУЧИЛОСЬ
Прогон 1 — буквальный повтор	тот же текст дважды	точное совпадение поймало мгновенно: заявка не записана, ответа не было; сам прогон технически успешен
Прогон 2 — перефразировка	та же пара с экрана сцены: «когда можно оплатить занятие?» → «как мне внести оплату за курс»	точное совпадение не поймало — дошло до модели; ответ — **«да»**

На прогоне 1 ничего не произошло молча при зелёном статусе — ровно то, ради чего дешёвый фильтр и стоит первым. На прогоне 2 модель ответила верно там, где оба механических подхода на этой паре дают ноль.

Механика и смысл — разные инструменты, не версии друг друга

Системный приём. Точное совпадение остаётся в финальном решении — но как первый дешёвый фильтр, экономящий 5 из 20 вызовов модели бесплатно, не как альтернатива LLM.

Дальше по курсу. На Семинаре 10 эта же механика — сравнение смысла заявок — расширяется до поиска ответа в базе знаний.

Нюансы проверки на повтор

1 **Интервал времени — параметр с ценой в обе стороны.** Слишком короткий интервал пропустит повтор, разнесённый на день-два. Слишком длинный начнёт путать разные последовательные обращения одного человека с повтором одного обращения.

1 **LLM-судья имеет собственную асимметрию ошибок**, не ноль (Wataoka, Takahashi, Ri — NeurIPS 2024 Safe GenAI Workshop, arXiv:2410.21819, self-preference bias): модель-судья систематически не нейтральна, даже отвечая формально «да» или «нет». Наш тест это подтвердил напрямую: 3 ложных срабатывания из 20, и не все объясняются совпадением слов в вопросах.

1 **«Что считать дублем» — продуктовое решение, не только техническое.** Иногда «тот же вопрос другими словами» стоит обработать как новое обращение — человек ждёт ответа срочнее второй раз. Не всякий технический повтор должен схлопываться в один в интерфейсе.

Когда точного совпадения достаточно

ТОЧНОГО СОВПАДЕНИЯ ДОСТАТОЧНО

повторы в основном буквальные, как первый вопрос Бориса

нужен бесплатный и мгновенный фильтр на весь поток заявок

классический векторный подход не стоит добавлять «на всякий случай» — измерение показало результат ХУЖЕ хэша, не тот же самый

ПРОВЕРКА СМЫСЛОМ НУЖНА СВЕРХУ

ожидаются переформулировки, как его второй вопрос

цена необнаруженного дубля выше цены одного вызова модели на спорные случаи

частота ошибок самой модели — измеренная цена (у нас 3 ложных срабатывания из 20), а не ноль, и её тоже нужно закладывать

ни хэш, ни классический векторный подход не решают переформулировку на русском языке целиком сами по себе

конкретный инструмент смысловой проверки — тоже выбор: дешёвая модель общего назначения (17/20 на нашем тесте) или специализированный сервис jev (20/20) — не угадывается по умолчанию, нужна своя проверка

Закрывает строку оси. Выглядело надёжным: заявка про тот же вопрос пришла снова — обработали как новую. Чем оказалось: без проверки на дубль каждое повторное обращение запускает цепочку с нуля; ни хэш, ни классический векторный подход не решают это целиком для живого русского текста — нужна модель, с её собственной, измеренной ценой ошибки.

Кейс 3. Одобрение возврата

«Автоматический возврат средств — до заданного порога суммы. Выше порога — только через руководителя.»

Версия 1: продолжение процесса классификации

Та же форма обращения, которой мы уже пользовались в предыдущих кейсах: человек описывает ситуацию своими словами — «прошу вернуть 2500 рублей за пропущенное занятие».

1. Человек пишет заявку свободным текстом — внутри него и просьба, и сумма.
2. Процесс, который в Семинаре 7 определял тему обращения, здесь идёт на шаг дальше: читает текст и сам решает — одобрить сумму автоматически или передать её руководителю (модель `qwen/qwen-2.5-7b-instruct`).
3. Решение «одобрить» — деньги автоматически уходят на карту клиента, без участия человека.
4. Решение «эскалировать» — заявка уходит руководителю, и дальше решает он.

На чём стоит доверие. Версия работает уже месяц. Никто не пожаловался, что одобрено то, что следовало эскалировать, или наоборот. Команда считает правило надёжным — и ровно на этом основании: ничего плохого не заметили.

Как добиться, чтобы решение о пороге не ошибалось

Как добиться, чтобы решение о пороге никогда не принималось неверно?

- написать правило яснее в промте
- дать модели инструмент-калькулятор для сравнения сумм
- поставить IF-узел после извлечения суммы
- сделать сумму явным числовым полем формы

Что даёт каждое из четырёх действий

ДЕЙСТВИЕ	КАК ДЕЛАЕТ	ПРЕИМУЩЕСТВО	НЕДОСТАТКИ
Написать правило яснее в промте	Переформулировать правило, явно развести «≤3000 → одобрить» и «>3000 → эскалировать»	Измерено: версия 1 ошибалась 9/101 на границе, уточнённая формулировка — **100/100** на расширенном наборе	Факт на конкретном наборе/модели, не структурная гарантия — решение всё ещё текстовое, не код; другая формулировка или модель может разойтись снова
Дать модели инструмент-калькулятор	Модель получает функцию сравнения чисел вместо сравнения «в уме» и сама решает, вызывать её или нет	В теории разделяет извлечение числа и сравнение на две операции	Модель сначала извлекает сумму из текста — тот же вероятностный шаг, затем сама решает, вызывать ли инструмент: вероятностность не исчезает, переезжает раньше. Измерено: **56/100** — хуже всех; калькулятор не умеет сравнений (11/100 отказов), 30/100 ответов не сводятся к слову, 9/100 ошибок разбросаны по диапазону
Поставить `IF`-узел после извлечения суммы	Сумму достаёт модель, сравнивает с порогом код — условие, не текстовый вызов	Устраняет вероятностность из точки решения. Измерено: **0/101** на расширенном наборе, включая граничные случаи	Сумму всё ещё извлекает модель — ошибка извлечения (не сравнения) этим узлом не закрыта, хотя на наборе её не нашлось
Сделать сумму явным числовым полем формы	Собирать сумму цифрами в поле заявки — не извлекать из текста вовсе	Устраняет неопределённость раньше: извлекать нечего — риска неверного извлечения нет	Работает только если форму можно менять; письмо/чат/другой канал всё равно требует извлечения из текста. Метрика — применимость, не точность: шага извлечения нет, точности тоже

Мостик дальше. Одно действие измеримо исправило конкретную находку, одно измеримо провалилось, одно устраняет риск структурно и подтверждено измерением, одно меняет вход раньше решения и не нуждается в метрике точности. Какое из них нужно именно здесь — решает не спор за столом, а измерение, которое вы уже видите в этой таблице: следующий такт показывает, как оно было устроено.

Проверка на практике, а не на веру

Зачем мерить, а не ждать жалобу. Жалоба приходит только если ошибка достаточно заметна и достаточно дорога, чтобы кто-то вернулся. Тихая ошибка — одобрена чуть большая сумма, эскалирована чуть меньшая — жалобы не вызовет никогда.

Исходные 51 (кластер у границы). Фиксированный seed, набор воспроизводим. 25 сумм near-or-below порога (1000–3000 руб., плотный кластер у самой границы: 2900, 2950, 2990, 2999, и ровно 3000 руб. — трижды намеренно). 25 сумм явно выше (3200–9000 руб., контрольная группа «очевидно эскалировать»). Плюс 1 ручной тест. Каждая сумма — внутри одного из четырёх реалистичных шаблонов фразы, не голым числом.

Новые 50 (сложнее первых). Три группы: tight border — 15 сумм в пределах ±20 руб. от порога, плотнее исходного кластера (2980…3020). Два числа — 20 текстов с отвлекающим числом (цена занятия, прошлый платёж) и целевым числом возврата в одном сообщении. Непрямая формулировка — 15 текстов, где сумма не названа явно, а выводится арифметикой («два занятия по 1500») или словом «около».

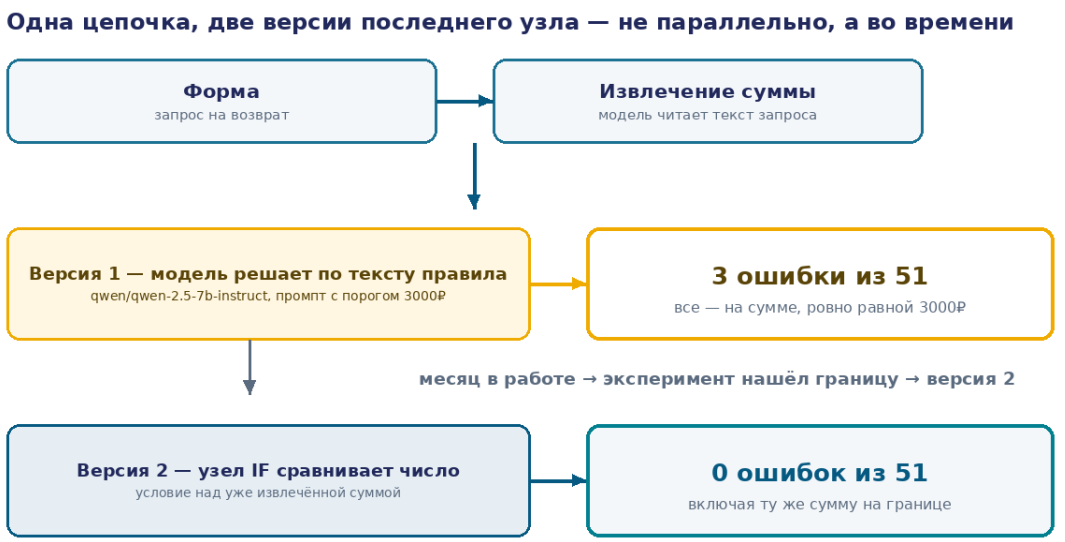
Итого 101 запрос. Критерий правильности — решение, прочитанное из `runData` настоящего прогона, сравнивается с ожиданием «одобрить, если сумма ≤ 3000 руб., иначе эскалировать» — тем же правилом, которое задаёт сам текст промпта.

Результат: 9 ошибок из 101, и все девять в одной точке

БЛОК ТЕСТА	ЗАПРОСОВ	ОШИБОК ВЕРСИИ 1	ОШИБОК ВЕРСИИ 2
Исходные, у границы и ниже (1000–3000 руб.)	25	**3** (сумма = 3000 руб.)	0
Исходные, явно выше (3200–9000 руб.)	25	0	0
Исходный ручной тест	1	0	0
Новые, у самой границы (±20 руб. от 3000 руб.)	15	0	0
Новые, два числа в одном тексте	20	0	0
Новые, непрямая формулировка (арифметика/«около»)	15	**6** (сумма = 3000 руб.)	0
Итого	**101**	**9**	**0**

Что решила модель на границе. На сумме ровно 3000 руб. — в любой из формулировок, прямой или арифметической, — модель версии 1 отвечала «эскалировать», хотя правило («не возвращай больше 3000 рублей») подразумевает, что сама граница — «можно одобрить». Все 9 ошибок без исключения — на этой одной сумме; ни одна ошибка не случилась ни на 2999 руб., ни на 3001 руб., ни где-либо ещё в диапазоне.

Версия 2: тот же путь, одна точка меняется

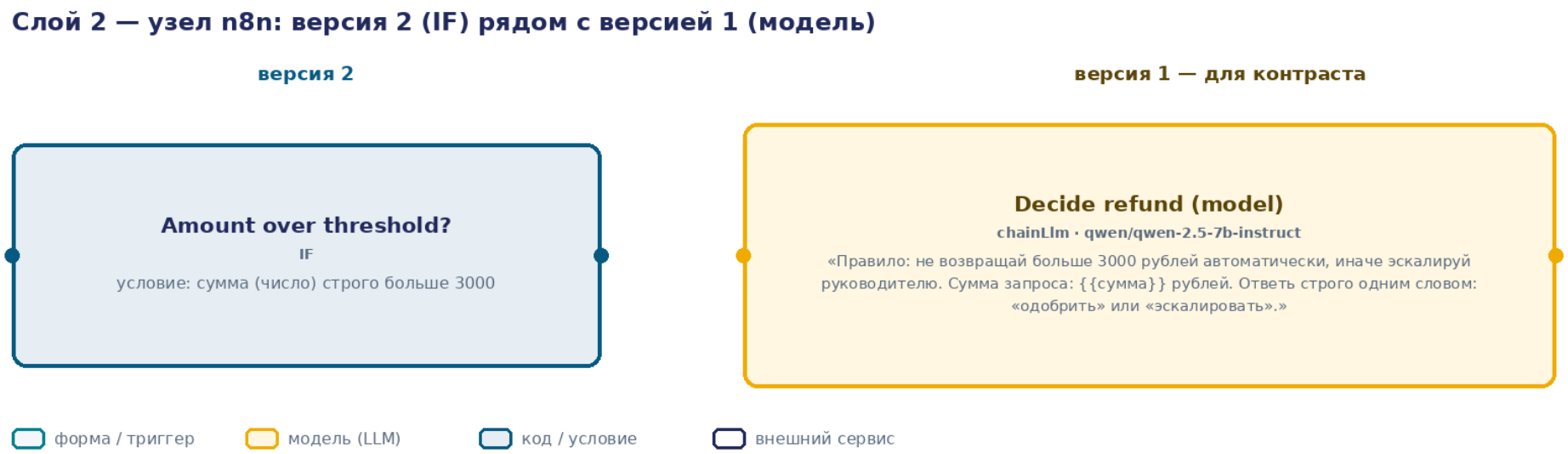


Версия 1 (было). Форма со свободным текстом → модель извлекает сумму → модель же решает по тексту правила → результат.

Версия 2 (стало, прямой ответ на находку эксперимента). Та же форма → модель извлекает сумму → `IF`-узел сравнивает число с порогом → результат.

Цепочка не перестроена. Разница — ровно в одном узле, том самом, где раньше принималось решение. Всё остальное — форма, извлечение суммы, дальнейшая маршрутизация заявки — осталось тем же.

Настройка: IF-узел против текста правила



Узел `IF` («Amount over threshold?»), версия 2. Условие: извлечённая сумма (число) строго больше 3000. Ветка «да» → эскалировать руководителю. Ветка «нет» → одобрить. Сравнение — операция над числом, не текст, который нужно понять.

Для контраста — промпт версии 1, тот же шаг. «Правило: не возвращай больше 3000 рублей автоматически, иначе эскалируй руководителю. Сумма запроса: {{сумма}} рублей. Ответь строго одним словом: «одобрить» или «эскалировать».» Здесь то же сравнение существует только как смысл внутри текста — у модели нет отдельной операции «сравнить числа», есть только одна операция «понять и ответить».

Разбор версий после теста

	ВЕРСИЯ 1 (ПРАВИЛО В ПРОМТЕ)	ВЕРСИЯ 2 (IF-УЗЕЛ)
Ошибок на 101 запросе	**9** — все на сумме 3000 руб.	**0**
На границе (3000 руб.)	«эскалировать», хотя правило подразумевает «можно одобрить»	«одобрить» — условие `> 3000` не допускает двойного толкования
Вне границы	верно во всех случаях, включая суммы в 20 руб. от границы и тексты с двумя числами	верно во всех случаях

Где решение должно быть кодом

Точная формулировка находки. Не «модель иногда вероятностно ошибается» — точнее: модель систематически промахивается именно на границе условия, там, где текст правила и его формальная интерпретация расходятся, а в остальном диапазоне безупречна.

Класс задачи. Решение, формализуемое как условие над числом, — не задача модели в точке принятия решения. Граничные значения — первое, что нужно проверить прогоном, прежде чем доверять любому похожему правилу.

Три нюанса, прежде чем применять вывод

1. **Инструмент-калькулятор у модели — не гарантия, и на практике измеримо хуже.** Вероятностность не исчезает, она переехала на шаг раньше: вызовет ли модель калькулятор и как передаст в него значения. Измерено реально: 56 из 100 — хуже всех остальных вариантов, с новыми отказами (смешанный алфавит в ответе, инструмент не умеет сравнений, ошибки разбросаны по всему диапазону).
2. **Порог X — единственный источник правды.** Если он живёт в двух копиях — тексте промпта и условии `IF` — расхождение этих копий — тот же класс проблемы, что рвущийся контракт между шагами.
3. **Модель-экстрактор всё ещё может ошибиться.** Вывод не «модель нигде не нужна», а «модель не должна быть точкой решения»; явное числовое поле формы убирает и эту часть риска — но только если форму самого запроса можно менять, иначе извлечение остаётся задачей модели.

Критерий: когда решение обязано быть кодом

Критерий. Решение обязано быть кодом, когда оно формализуется как условие над уже структурированным значением. Ещё лучше — устранить саму потребность в извлечении, если источник данных можно сделать структурированным с самого начала.

Строка оси закрыта. Решение, оставленное модели, обходило правило в измеренной доле прогонов — девять из ста одного, и ровно на границе условия. Решение, отданное алгоритмическому гейту, — никогда.

1 Zillow Offers: алгоритмическая закупка жилья

«Мы определили, что непредсказуемость прогнозирования цен на жильё намного превышает то, что мы ожидали… наблюдаемая частота ошибок оказалась гораздо более волатильной, чем мы вообще считали возможным.»

— Рич Бартон, CEO Zillow, 2 ноября 2021 года, объявляя закрытие бизнеса Zillow Offers

Zillow — крупнейший агрегатор объявлений о недвижимости в США. С 2018 года компания вела направление Zillow Offers, другое название — iBuying: алгоритмическая модель Zestimate оценивала дом по открытым данным, компания покупала его почти мгновенно по этой цене, делала минимальный ремонт и быстро перепродавала, зарабатывая на разнице между покупкой и продажей.

Списание запасов — $304,4 млн за 3-й квартал 2021 года, по итогам всего года — $407,9 млн. Сокращение штата — ~25%, это ~2000 человек. В том же квартале Zillow купила 9680 домов, продать успела лишь 3032 — средний убыток на каждый проданный дом

Как решение о покупке работало на самом деле



Zestimate (алгоритмическая оценка) → решение о покупке без обязательного гейта выше порога риска → покупка исполняется → без короткого периода мониторинга точности (часы-дни) → ошибка накапливается незаметно → видна только в квартальной отчётности перед SEC.

Чего не хватало в контуре решения Zillow

Чего не хватало в контуре принятия решения?

- **откалибровать модель точнее**
- **поставить runtime-мониторинг дрейфа с автоматическим порогом остановки**
- **привязать решение о покупке выше определённой суммы риска к конкретному человеку-андеррайтеру**
- **заранее знать, что рынок жилья поведёт себя непредсказуемо**

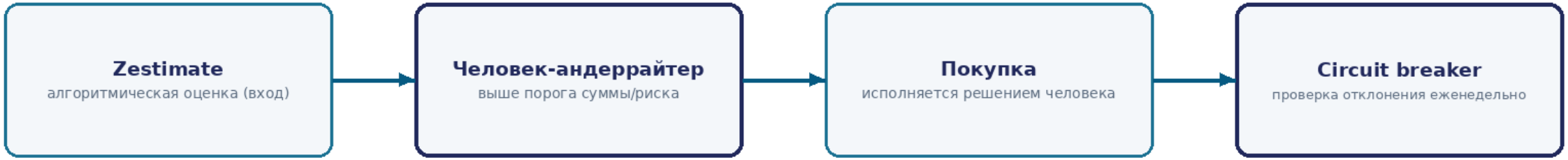
Что решает каждый вариант

ВАРИАНТ	ЧТО ДЕЛАЕТ	ЧЕГО НЕ ДЕЛАЕТ
откалибровать модель точнее	повышает точность на стабильных исторических данных	не относится к находке: модель и так была откалибрована корректно — ошиблась не калибровка, а режим рынка, которого не было в обучающей истории
runtime-мониторинг дрейфа с автоматическим порогом остановки	ловит резкий скачок частоты ошибки за часы или дни, пока убыток ещё маленький	не был поставлен на реальном контуре закупки: единственным контролем оставалась квартальная отчётность перед SEC — архив, а не период наблюдения
привязать решение выше порога риска к человеку-андеррайтеру	возвращает решение с крупной ставкой человеку именно в точке принятия решения	не было сделано структурно: алгоритм решал о покупке дома сам, вне зависимости от суммы риска
заранее знать, что рынок поведёт себя непредсказуемо	—	не инженерное решение, а пожелание: волатильность рынка — внешний факт, которым нельзя управлять на стороне модели

Решающее. Модель не подвела — подвёл контур вокруг неё: не было ни короткого периода мониторинга, ни структурной привязки крупных решений к человеку. Решение, управляющее сотнями миллионов долларов, оставалось вероятностным там, где давно стоило стать гейтированным.

Архитектурная альтернатива: андеррайтер и circuit breaker

Альтернатива с гейтом: человек выше порога риска + circuit breaker



Решение выше порога риска — у человека, не у модели.

Окно контроля — неделя, не квартал.

Zestimate (алгоритмическая оценка, вход, не решение) → человек-андеррайтер принимает финальное решение о покупке выше порога суммы/риска → explicit circuit breaker на отклонение фактической цены от прогнозной, проверяемый еженедельно или чаще — не поквартально.

Когда снижать объём автоматизации

СНИЖАТЬ ОБЪЁМ АВТОМАТИЧЕСКИХ РЕШЕНИЙ

- цена одной ошибки — десятки или сотни тысяч долларов реального капитала
- внешние условия показывают растущую волатильность относительно того режима, на котором модель обучена
- решение выше порога суммы или риска не привязано ни к одному конкретному человеку
- единственный контроль качества — отчётность раз в квартал

МОЖНО ПРОДОЛЖАТЬ МАСШТАБИРОВАТЬ

- цена одной ошибки — один неприятный разговор с клиентом, не капитал
- рынок и данные стабильны, режим совпадает с тем, что модель видела при обучении
- решение выше порога структурно идёт через человека, и есть работающий circuit breaker
- есть короткий период мониторинга (часы-дни) с автоматическим порогом остановки

Предел. Если стоимость одной ошибки модели — десятки тысяч долларов реального капитала, а внешние условия демонстрируют растущую волатильность, снижайте объём автоматизированных решений до подтверждения устойчивости модели к новому режиму — не масштабируйте «пока не увидим цифры за квартал».

Та же мысль, что в «Одобрении возврата», на другом масштабе. Там гейтом было решение по одной заявке на возврат, ценой ошибки — неловкий разговор с клиентом. У Zillow гейтом стала каждая покупка дома, а ценой пропущенного порога — реализованный убыток в сотни миллионов долларов и примерно четверть всего штата. Порог существовал в обоих случаях; алгоритмическим гейтом он не стал ни там, ни там, пока это не проверили.

Карточка в Telegram

Журнал прогона — зелёный, узел отправки вернул успешный ответ. У координатора в Telegram — ничего не пришло.

Потребность маркетинга и первая версия

Учебный центр уже отправляет координатору текстовое уведомление о новой заявке (кейс «Уведомления»). Маркетинг просит: вместо сухого текста — карточка, текст поверх картинки, в том же канале.

1. Собрали цепочку: текст уведомления готовится тем же приёмом подстановки значений, что в «Уведомлениях»; следующий шаг забирает картинку по ссылке; последний шаг отправляет картинку в Telegram с этим текстом как подписью.
2. На тестовых картинках цепочка работает без замечаний: карточка собирается и уходит, координатор видит её в Telegram.
3. Маркетинг присылает первую настоящую карточку для реальной рассылки — скан высокого разрешения прямо с телефона, заметно больше тестовых картинок.
4. Прогон завершается успешно: журнал n8n зелёный, узел отправки получил от Telegram API положительный ответ. У координатора в Telegram карточки нет — либо она не пришла вовсе, либо пришла обрезанной.

Цена, которую не видно в журнале. Это не отказ узла и не ошибка вызова — такие мы уже умеем ловить. Это тихая потеря результата при формально успешном прогоне, и до сих пор у нас не было повода проверять бинарные данные иначе, чем сам факт, что они сформировались.

Что должно быть верно, прежде чем карточка считается готовой. Два класса требований, разных по природе:

КЛАСС ТРЕБОВАНИЯ	ЧТО ПРОВЕРЯЕТСЯ
Механическое	размер файла, формат, разрешение — у файла есть численные и перечислимые свойства
Смысловое / по содержанию	картинка на самом деле показывает то, что нужно — не битый файл-заглушка, не чужое изображение, не пустой кадр

Как проверить, что картинка соответствует требованиям

Как проверить, что картинка соответствует требованиям?

- проверить размер и формат файла перед отправкой — программно
- проверить, что картинка открывается без ошибок — программно
- проверить, что на картинке действительно то, что нужно показать — моделью с распознаванием изображений
- ничего не проверять — доверять тому, что узел отправки сам разберётся

Какая проверка закрывает какое требование

КАРТОЧКА	КАКОЕ ТРЕБОВАНИЕ ЗАКРЫВАЕТ	ЗАКРЫТО СЕГОДНЯ?
проверить размер и формат файла перед отправкой	механическое	да — код, детерминированно, до отправки
проверить, что картинка открывается без ошибок	механическое (тот же класс)	да — тем же способом
проверить, что на картинке действительно то, что нужно	смысловое / по содержанию	нет — в цепочке сегодня нет шага, который это умеет
ничего не проверять	ни одно	нет — это и случилось в сцене кейса

Технически исправный файл (правильный размер, правильный формат) всё равно может показывать не то, что нужно — например если узел получения картинки получил страницу ошибки хостинга вместо самой картинки: формат и размер пройдут механическую проверку, а содержимое будет не тем.

Дальше — как первые два пункта реализуются технически: архитектура без привязки к инструменту, а следом настоящая конфигурация в n8n.

Архитектура: два класса требований к файлу

Два класса требований к картинке — одна и та же ось «модель или код»

Механическое

- размер файла
- формат
- разрешение

код проверяет — у файла есть измеримые свойства

Смысловое / по содержанию

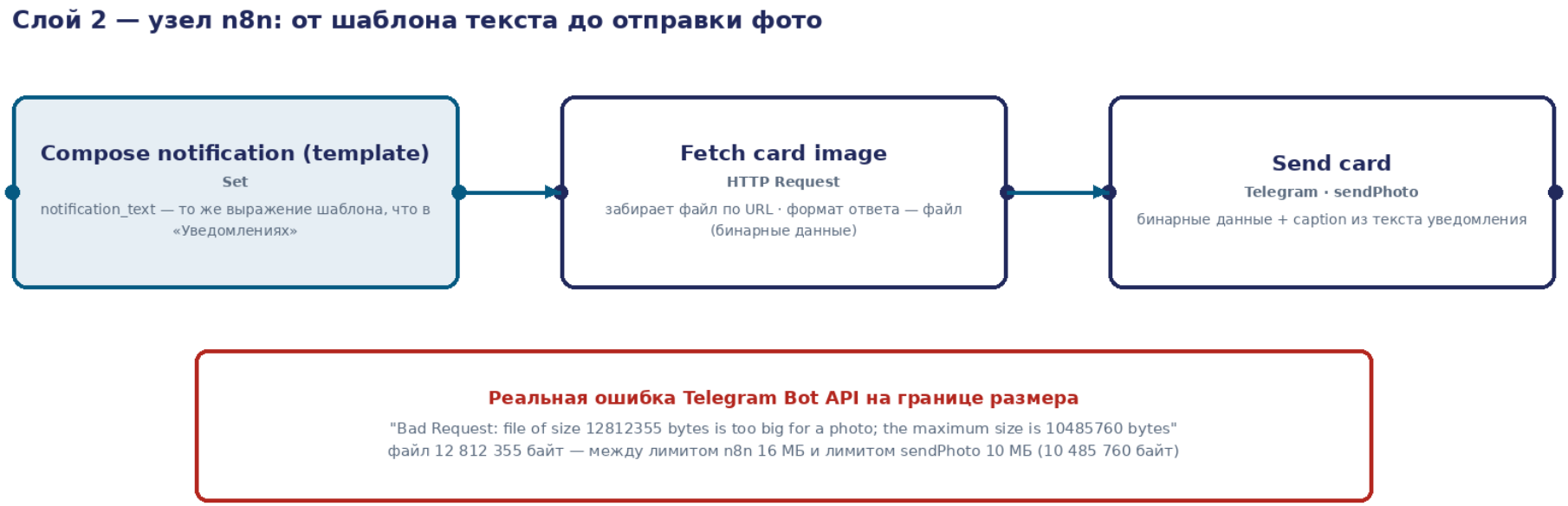
картинка на самом деле показывает то, что нужно — не битый файл-заглушка, не чужое изображение

нужна модель с распознаванием
не покрыто сегодня

Технически исправный файл всё равно может показывать не то, что нужно.

Файл → механическое требование (размер, формат, разрешение) → проверяет код, детерминированно, до отправки. Файл → смысловое требование (показывает ли нужное) → проверяет модель с распознаванием изображений — сегодня этого шага в цепочке нет.

Цепочка в n8n и граница размера, проверенная настоящим вызовом



ЧТО	БАЙТ
Проверенный файл	**12 812 355** (~12,8 МБ)
Лимит Telegram на фото	**10 485 760** (10 МБ)

Ответ Telegram Bot API: "Bad Request: file of size 12812355 bytes is too big for a photo; the maximum size is 10485760 bytes"

Генерация карточки: текст внутри изображения



Архитектурная оценка, не узел на стенде: генерация картинки сегодня в цепочке не собрана.

Текст уведомления и требование «текст поверх картинки» → специализированная генерирующая модель (текст → изображение) → готовая карточка, текст уже встроен в пиксели → отправка в Telegram как фото.

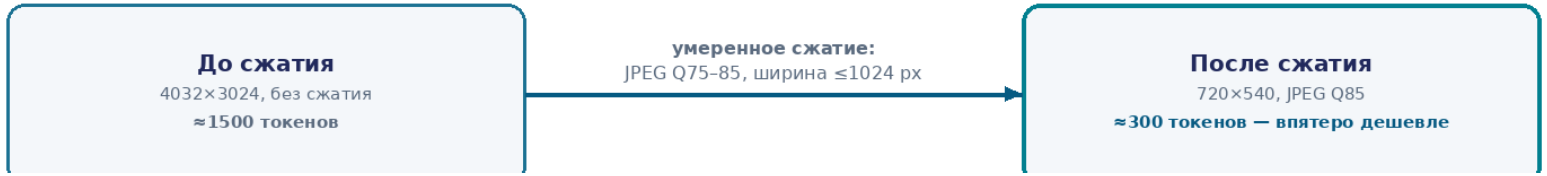
Стоимость генерации одной картинки — gpt-image-1.5 (OpenAI), 1024×1024: низкое качество $0,009 · среднее $0,034 · высокое $0,133 за картинку. Для уведомления подойдёт среднее — ≈$0,034 за карточку. Примечание. Генерируемые картинки приходят большого стандартного размера, слабо сжатые — перед отправкой в Telegram нужна перекодировка в более лёгкий формат, тот же лимит `sendPhoto` 10 МБ, что мы уже измерили.

Порядок проверки и вопрос сжатия

Механическая проверка → модель содержимого, и вопрос сжатия



Сжатие перед отправкой в модель-проверяющую



ДЕЙСТВИЕ	КАК ДЕЛАЕТ	ПРЕИМУЩЕСТВО	НЕДОСТАТКИ
Без сжатия	как есть (4032×3024)	максимум деталей	дорого — ≈1500 токенов
Умеренное сжатие	Q75–85, ≤1024 px	≈300 токенов, впятеро дешевле	режет мелкий текст у предела
Агрессивное сжатие	низкое качество, < ~1024 px	ещё дешевле	текст и детали не распознать

Контракт на бинарных данных и граница вводного уровня

Бинарные данные в n8n — отдельная модель данных, не просто ещё одно значение поля. Контракт между шагами, который в «Уведомлениях» рвался на тексте (узел не падал, результат тихо не тот), здесь рвётся на формате и размере — по той же причине: формально успешный прогон ничего не говорит о том, дошёл ли результат до получателя.

НА ЭТОМ УРОВНЕ ДОСТАТОЧНО	СЛЕДУЮЩИЙ УРОВЕНЬ — СЕМИНАР 10
один файл, один известный лимит, одна отправка	хранение и поиск среди множества файлов
проверка размера и формата перед отправкой	обработка содержимого файла моделью (поиск по смыслу изображений)
явное условие в цепочке на каждый шаг с бинарными данными	векторное хранилище и индексация вложений

Когда бинарные данные требуют отдельной заботы

НУЖНО ВСЕГДА

проверка размера и формата перед отправкой любого файла или изображения в цепочке

знание лимита канала доставки, а не только лимита самого оркестратора

НЕ ТРЕБУЕТ ЭТОГО ЗАНЯТИЯ

хранение множества файлов

поиск среди вложений и обработка их содержимого моделью

Четвёртая строка оси закрыта. Выглядело надёжным: картинка собралась и ушла в Telegram. Чем оказалось: до тех пор, пока помещается в лимит — большой файл уходит не полностью или не уходит вовсе, а прогон при этом остаётся зелёным.

1. **Собственный лимит n8n по умолчанию — 16 МБ** на бинарные данные. Для фотографии с текстом этого обычно достаточно, но скан высокого разрешения или фото прямо с телефона — не всегда.
2. **Лимиты Telegram — отдельный, более узкий потолок**, и они разные для фото и для документа. То, что прошло мимо лимита n8n, может всё равно не пройти мимо лимита канала доставки — два независимых порога, не один.
3. **Ручной прогон с бинарными данными потребляет заметно больше памяти**, чем автоматический, — разница заметнее, чем на текстовых полях, и стоит иметь это в уме при отладке такого узла вживую.

Четыре строки оси: что оказалось

ВЫГЛЯДИТ НАДЁЖНЫМ	ЧЕМ ОКАЗАЛСЯ
сообщение ушло координатору, цепочка короче и дешевле без LLM	LLM здесь был просто лишним шагом — то же самое сделал шаблон, без модели и без риска галлюцинации (**«Уведомления»**)
заявка про тот же вопрос пришла снова — обработали как новую	без проверки на дубль каждое повторное обращение запускает цепочку с нуля; ни точное совпадение, ни похожесть по словам не решают это целиком — для переформулировки нужна проверка смыслом (**«Проверка на повтор»**)
в промте написано «не возвращай больше суммы X»	решение оставалось вероятностным: модель обходила правило в измеренной доле прогонов, ровно на границе условия; алгоритмический гейт — никогда (**«Одобрение возврата»**, подтверждено на масштабе компании — Zillow)
картинка собралась и ушла в Telegram	до тех пор, пока помещается в лимит — большой файл уходит не полностью или не уходит вовсе, а прогон всё равно зелёный (**«Карточка в Telegram»**)

Где в цепочке решение принимает модель, а где должен — код, и что молчит, когда эту границу не назвали явно?

Что уносим с занятия

ФАЙЛ	ПРОДОЛЖЕНИЕ `student.json`, С КОТОРЫМ ВЫ ЗАКОНЧИЛИ ПРОШЛОЕ ЗАНЯТИЕ
что добавилось	уведомление координатору шаблоном, а не моделью · проверка на повтор в два уровня · решение о возврате условием в коде, а не правилом в тексте · доставка картинки с проверкой размера перед отправкой
как поставить	тот же импорт, что и в прошлый раз: меню сценария → Import from File
что нужно	свой n8n и ключ провайдера модели, как и раньше, плюс бот и токен в Telegram для уведомления и картинки
что делать	прогнать несколько заявок, включая повтор и крупную картинку, и посмотреть, где какое из четырёх решений ведёт себя не так, как на экране

Что дальше

Сейчас: любой вход → один и тот же путь через цепочку

Дальше: вход типа А → один путь · вход типа Б → другой · вход, не подошедший ни под один тип → свой путь по умолчанию

Семинар 9: условие против переключателя · классификация как отдельный шаг цепочки · что делать со входом, который не подошёл ни под одну ветку · как ветки, которые выглядят параллельными на холсте, выполняются на самом деле